Name:

Stephen Ugbana

Project Title:

Social Media Sentiment Analysis and Topic Modelling Using Python and NLP

About This Project

This project analysed Reddit channels and comments about Taylor Swift to understand her audience engagement, discussion themes, and sentiment, for evidence-based PR decision-making. Public Reddit data were collected via RSS feeds and processed with **Python** for data cleaning, preprocessing, exploratory analysis, and text mining. **VADER sentiment analysis** was then used to measure audience sentiment, while **LDA topic modelling** was used to identify dominant themes in the discussions.

Skills

Python · Data Collection · RSS Data Extraction · Data Cleaning & Preprocessing · Exploratory Data Analysis · Text Mining · Sentiment Analysis · Topic Modelling · Natural Language Processing · Pandas · NLTK · Scikit-learn · Matplotlib · Data Visualisation · Statistical Analysis · Data Interpretation · Business/PR Insights · REST API · Git & GitHub

GitHub Link

https://github.com/StephenCOUgbana/taylor_swift_reddit_analysis

TABLE OF CONTENTS

1. EXECUTIVE SUMMARY

1.1. Summary and Overview

This report analyses Taylor Swift's Reddit presence from the perspective of a data scientist working for a PR company. Taylor Swift was selected because she has a large, active and highly engaged fan community. Reddit was chosen because it captures long, fan discussions, debates, and emotional reactions. Data from May 2025 to May 2026 was collected from Public Reddit RSS feeds, then it was cleaned and analysed using exploratory and text-mining methods.

1.2. Findings and Results

- Comments stayed active and increased in March, April and May 2026, suggesting event-driven discussion linked to fan debates, tours or news coverage (Figure 9).
- The most active posts focused on albums, vinyl, reviews and fan reflections (Table 3/Figure 11).
- Frequent words such as "song", "album", "music", "fan", "era", "track", "ttpd" (The Tortured Poets Department) and "tour" show that discussion is strongly music-centred (Figure 13).
- Sentiment was mainly positive, with 2,300 positive comments, representing 64.75% of the dataset (Table 4/Figure 19).
- LDA topic modelling showed that albums and discography were the largest themes, with 1,803 comments, while Tour, concerts and live events had the highest average sentiment at 0.382 (Table 8/Figure 26).

1.3. Recommendations and Suggested PR Actions

- Prioritise music storytelling.
- Build album campaigns.
- Amplify tour nostalgia.
- Encourage fan reflections on songs.
- Monitor broader celebrity discussions for reputational risks.

2. DATA

2.1. Data Collection Process

The data was collected from Taylor Swift's Reddit community using publicly accessible Reddit RSS feeds, such as:

- https://www.reddit.com/r/TaylorSwift/new/.rss
- https://www.reddit.com/r/TaylorSwift/hot/.rss
- https://www.reddit.com/r/TaylorSwift/top/.rss?t=year

I also used RSS search feeds containing Taylor Swift-related terms such as “Taylor Swift”, “Eras Tour”, “album”, “ttpd”, “Midnights”, “Reputation”, “Grammy”, “ticket”, “music video”, and “performance”.

These RSS feeds and search terms were chosen because they represent major areas of Taylor Swift discussion, including music releases, tours, fan reactions, awards, and media events.

The raw data collection produced three main CSV files:

- taylor_swift_reddit_rss_posts_raw.csv
- taylor_swift_reddit_rss_comments_raw.csv
- taylor_swift_reddit_rss_raw_dataset.csv

The metrics from the raw dataset are:

Table 1: Raw data metrics

Metric	Value
Number of rows in the raw dataset	4,498
Number of posts collected	250
Number of comments collected	4,498
Raw date range	15 May 2025 to 12 May 2026

Also, presented below is a screenshot of the raw Reddit RSS dataset

Figure 1: Raw Reddit dataset

Furthermore, the following images show a few sections of code developed to read the Reddit RSS feeds mentioned above.

```
import feedparser
import pandas as pd
import numpy as np
import time
import hashlib
import re
from bs4 import BeautifulSoup
from urllib.parse import quote_plus
from datetime import datetime
```

```
# ============================
# PROJECT SETTINGS
# ============================

SUBREDDIT = "TaylorSwift"

SEARCH_TERMS = [
    "Taylor Swift",
    "Eras Tour",
    "album",
    "The Tortured Poets Department",
    "Midnights",
    "Reputation",
    "Grammy",
    "ticket",
    "music video",
    "performance"
]

# RSS collection limits
MAX_POSTS_TOTAL = 250
MAX_COMMENTS_PER_POST = 30

# Be polite to Reddit RSS
REQUEST_PAUSE_SECONDS = 1.0

USER_AGENT = "MSc Web and Social Media Analytics Coursework - Taylor Swift Reddit RSS"
```

Figure 2: Code used to set up the RSS data collection

```
# ============================
# CREATE RSS FEED URLS
# ============================

feed_urls = []

# Main subreddit feeds
feed_urls.append(f"https://www.reddit.com/r/{SUBREDDIT}/new/.rss")
feed_urls.append(f"https://www.reddit.com/r/{SUBREDDIT}/hot/.rss")
feed_urls.append(f"https://www.reddit.com/r/{SUBREDDIT}/top/.rss?t=year")

# Search feeds inside the subreddit
for term in SEARCH_TERMS:
    encoded_term = quote_plus(term)
    feed_urls.append(
        f"https://www.reddit.com/r/{SUBREDDIT}/search.rss?q={encoded_term}&restrict_sr=1&sort=new&t=year"
    )

# Add old.reddit fallback versions
old_feed_urls = [url.replace("https://www.reddit.com", "https://old.reddit.com") for url in feed_urls]

all_feed_urls = feed_urls + old_feed_urls

print("Number of RSS feed URLs prepared:", len(all_feed_urls))
for url in all_feed_urls[:5]:
    print(url)
```

Figure 3: Code used to create the RSS feed URLs

```python
# ============================
# COLLECT COMMENTS FROM COMMENT RSS FEEDS
# ============================

comment_rows = []
seen_comment_links = set()

for idx, post_row in posts_df.iterrows():
    post_link = post_row["post_link"]
    candidates = make_comment_rss_candidates(post_link)

    comments_for_this_post = 0

    for comment_feed_url in candidates:
        feed = parse_rss(comment_feed_url)

        if len(feed.entries) == 0:
            continue

        for entry in feed.entries:
            comment_link = safe_get(entry, "link", "")

            if comment_link in seen_comment_links:
                continue

            seen_comment_links.add(comment_link)

            title = clean_text_basic(safe_get(entry, "title", ""))
            summary = clean_text_basic(html_to_text(safe_get(entry, "summary", "")))
            author = safe_get(entry, "author", "unknown")

            # Skip entries that are too short to be useful
            if len(summary) < 10 and len(title) < 10:
                continue

            comment_rows.append({
                "source_platform": "Reddit",
                "subreddit": SUBREDDIT,
                "post_title": post_row["post_title"],
                "post_link": post_link,
                "comment_title": title,
                "comment_body": summary,
                "comment_author_id": anonymise_author(author),
                "comment_author_raw": author,
                "comment_link": comment_link,
                "comment_published": safe_get(entry, "published", ""),
                "comment_datetime": get_entry_datetime(entry),
                "comment_rss_url": comment_feed_url
            })

            comments_for_this_post += 1

            if comments_for_this_post >= MAX_COMMENTS_PER_POST:
                break

        if comments_for_this_post >= MAX_COMMENTS_PER_POST:
            break

        time.sleep(REQUEST_PAUSE_SECONDS)

    if idx % 10 == 0:
        print(f"Processed {idx + 1}/{len(posts_df)} posts. Comments collected so far: {len(comment_rows)}")

comments_df = pd.DataFrame(comment_rows)

print("Number of Reddit comments collected from RSS:", len(comments_df))
comments_df.head()
```

Figure 4: Code used to collect comments from the RSS feeds

The code, in its entirety, loops through the above-mentioned RSS URLs, parses each feed, extracts relevant fields, and stores them in a structured dataframe.

The methods used for the data collection process are justified because they provide a practical, transparent, and ethical way to collect public Reddit discussions.

2.2. Data Cleaning and Pre-processing

The raw dataset generated required preprocessing before analysis because social media text often contains noise. This included URLs, punctuation, emoji, HTML artefacts, repeated content, empty comments, deleted comments, and very short comments that were not useful for text mining.

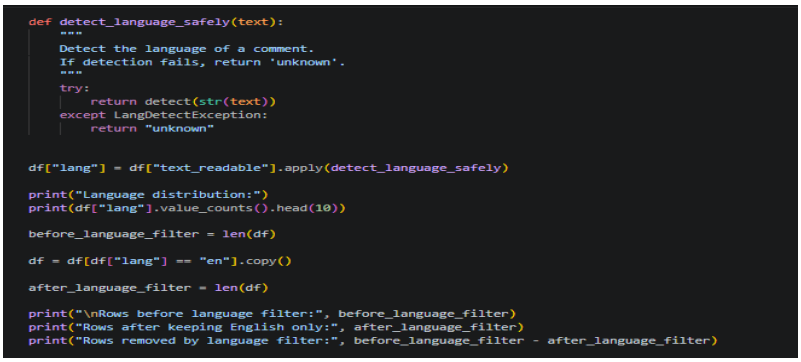
The pre-processing done to address the data quality and formatting issues was:

- removed deleted, removed, empty, and unusable comments
- removed duplicates
- filtered very short comments
- converted text to lowercase
- removed URLs and Reddit/user artefacts
- removed punctuation and non-informative symbols
- removed stopwords
- lemmatised words to reduce them to base form
- created a cleaned_text column for modelling
- created raw_word_count, raw_char_count, clean_word_count, and clean_char_count fields

The following images show a few sections of the code developed to perform data cleansing and preprocessing on the raw data generated.

```
def normalise_raw_text(text):
    """
    Convert RSS/HTML text into readable plain text.
    This removes HTML entities and extra spacing.
    """
    if pd.isna(text):
        return ""

    text = str(text)
    text = html.unescape(text)
    text = BeautifulSoup(text, "html.parser").get_text(" ", strip=True)
    text = text.replace("\u200b", " ").replace("\xa0", " ")
    text = re.sub(r"\s+", " ", text).strip()

    return text


df = raw_df.copy()

df["text_readable"] = df["text_for_analysis"].apply(normalise_raw_text)

df[["text_for_analysis", "text_readable"]].head(10)
```

Figure 5: Code used to convert RSS/HTML text into plain text



```
def detect_language_safely(text):
    """
    Detect the language of a comment.
    If detection fails, return 'unknown'.
    """
    try:
        return detect(str(text))
    except LangDetectException:
        return "unknown"


df["lang"] = df["text_readable"].apply(detect_language_safely)

print("Language distribution:")
print(df["lang"].value_counts().head(10))

before_language_filter = len(df)

df = df[df["lang"] == "en"].copy()

after_language_filter = len(df)

print("\nRows before language filter:", before_language_filter)
print("Rows after keeping English only:", after_language_filter)
print("Rows removed by language filter:", before_language_filter - after_language_filter)
```

Figure 6: Code for comment language detection

```python
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

# Add project-specific stopwords.
# These words are common in the dataset but may not be useful for interpretation.
custom_stopwords = {
    "taylor", "swift", "reddit", "comment", "comments",
    "link", "submitted", "amp", "https", "http", "www",
    "like", "just", "really", "would", "could", "also"
}

all_stopwords = stop_words.union(custom_stopwords)


def clean_for_text_mining(text):
    """
    Clean Reddit comment text for text mining.
    """
    text = str(text)

    # Expand contractions: don't -> do not
    text = contractions.fix(text)

    # Lowercase
    text = text.lower()

    # Remove URLs
    text = re.sub(r"http\S+|www\S+", " ", text)

    # Remove Reddit usernames and subreddit mentions
    text = re.sub(r"u/\w+", " ", text)
    text = re.sub(r"r/\w+", " ", text)

    # Remove punctuation, numbers and symbols
    text = re.sub(r"[^a-z\s]", " ", text)

    # Remove extra spaces
    text = re.sub(r"\s+", " ", text).strip()

    # Tokenise using simple split
    words = text.split()

    # Remove stopwords, very short words, and lemmatise
    words = [
        lemmatizer.lemmatize(word)
        for word in words
        if word not in all_stopwords and len(word) > 2
    ]

    return " ".join(words)


df["cleaned_text"] = df["text_readable"].apply(clean_for_text_mining)

df[["text_readable", "cleaned_text"]].head(10)
```

Figure 7: Code used to clean comments for text mining.

2.3. Final Cleaned Dataset

After preprocessing, the final cleaned dataset contained 3,552 comments, 164 unique Reddit posts, and 2,016 unique anonymised authors. The author's field was anonymised to protect user privacy and prevent the identification of individual Reddit users. Below is a table showing the summary statistics for the final cleansed dataset.

Table 2: Cleaned data metrics

Metric	Value
Final number of cleaned comments	3,552
Number of unique Reddit posts	164
Number of unique post titles	164
Number of unique anonymised authors	2,016
Average raw comment length	196.82 characters
Median raw comment length	106 characters
Average raw word count	35.89 words
Median raw word count	19 words
Shortest raw comment length	11 characters
Longest raw comment length	16,324 characters
Shortest raw word count	3 words
Longest raw word count	2,715 words

These summary statistics show that Taylor Swift's Reddit community is broad, with over 2000 authors contributing to discussions. The median comment length of 19 words suggests that many comments are short reactions, while the maximum of 2,715 words shows that some users write highly detailed reflections. This is valuable PR for analysis as it shows both quick audience reactions and deeper fan discussion.

This is a screenshot of the cleaned dataset

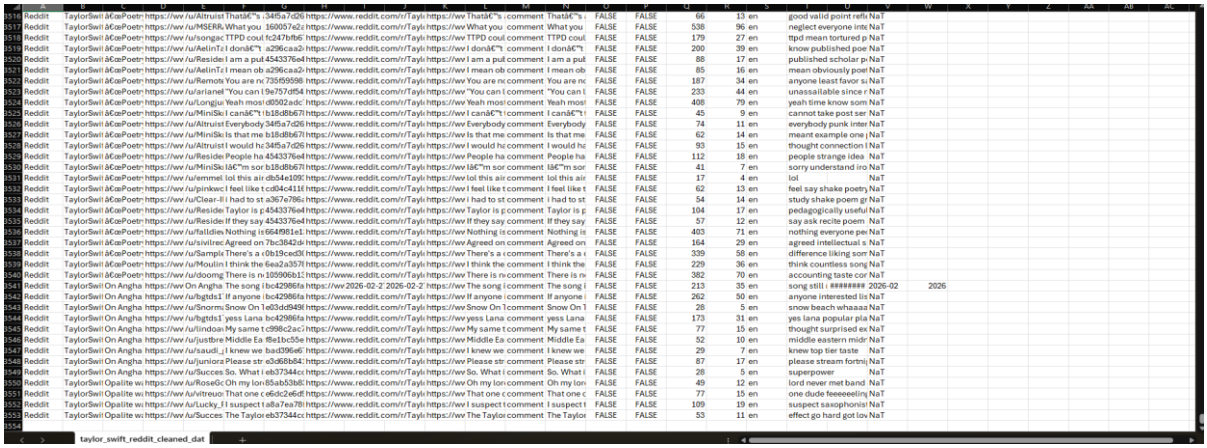


Figure 8: Cleaned Reddit dataset

3. EXPLORATORY DATA ANALYSIS

Exploratory data analysis (EDA) was conducted to understand activity patterns, engagement themes, and participation behaviour within the cleaned dataset. The analysis focused on five areas: comment activity over time, top discussion posts, frequent words, comment length, and active contributors.

3.1. EDA 1: Taylor Swift Reddit Comments Over Time

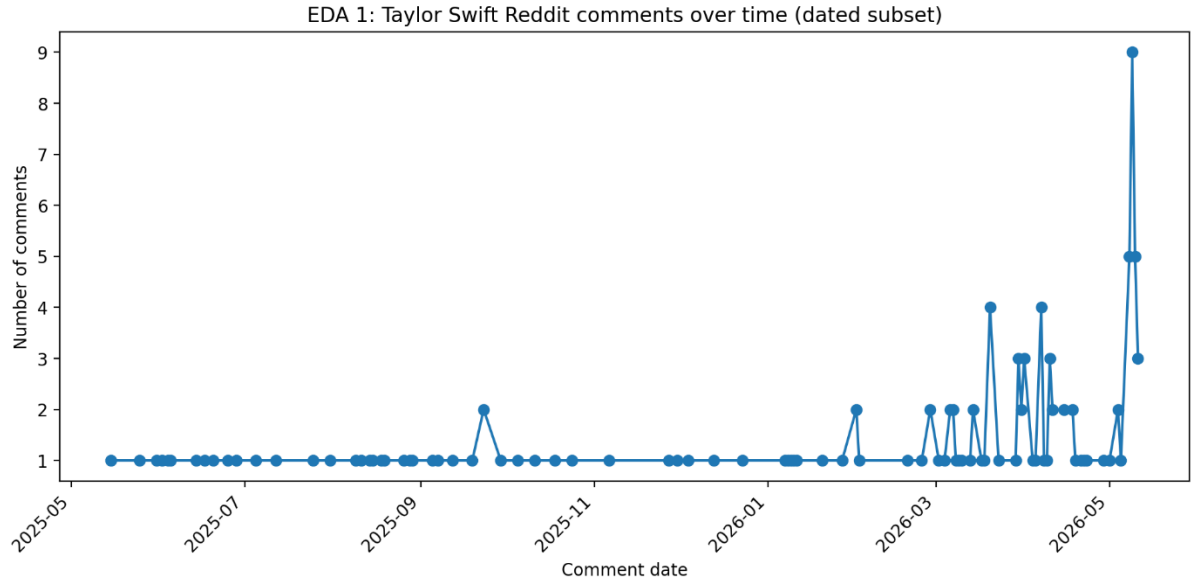


Figure 9: Taylor Swift Reddit comments over time

The comments span from 15 May 2025 to 11 May 2026. The peak was on 9 May 2026 with 9 comments, and 8-10 May 2026 each had 5 comments. Monthly totals rose towards the end of the collection period, with 25 comments in March and April 2026, and 26 in May 2026. PR-wise, activity increased from March to May 2026, indicating the discussion is likely event-driven and tied to fan debates, tours, and news coverage.

```
# ============================================================
# Exploratory Visual 1: Comments over time
# ============================================================

if len(dated_df) > 0:
    daily_comments = dated_df.groupby("comment_date_parsed").size().reset_index(name="comment_count")
    daily_comments.to_csv("outputs/eda_1_comments_over_time.csv", index=False)

    plt.figure(figsize=(10, 5))
    plt.plot(daily_comments["comment_date_parsed"], daily_comments["comment_count"], marker="o")
    plt.title("EDA 1: Taylor Swift Reddit comments over time (dated subset)")
    plt.xlabel("Comment date")
    plt.ylabel("Number of comments")
    plt.xticks(rotation=45, ha="right")
    save_current_figure("eda_1_comments_over_time")

    display(daily_comments.tail(15))
else:
    print("No parsed comment dates are available, so this time-series visual cannot be created.")
```

Figure 10: Code used to analyse comments over time

3.2. EDA 2: Top Reddit Posts by Number of Comments

The table below shows the top Reddit posts by the number of comments in the dataset.

Table 3: Reddit Posts with the highest comments

Rank	Post title	Comments
1	Do you think Taylor will repress her first six albums on vinyl?	30
2	TLOAS album would have been liked more if this album had multiple pre-album release singles with mvs to set the narrative.	30
3	Taylor Swift Honors Dad Scott with $1 Million Donation to American Heart Association After Bypass Surgery (Exclusive)	30
4	Hits Different in the context of TTPD	30
5	new (or newly serious) swifties: do you have any “swiftie regrets”?	30
6	Why Normal Music Reviews No Longer Make Sense for Taylor Swift	30
7	Think back to Reputation and your first listen: did you freak out with such a banger track 1	30
8	Theory Megathread: May 2026	30
9	Got a bit emo over Bigger Than The Whole Sky, and it led to an epiphany	29
10	Taylor song you want another to cover	29

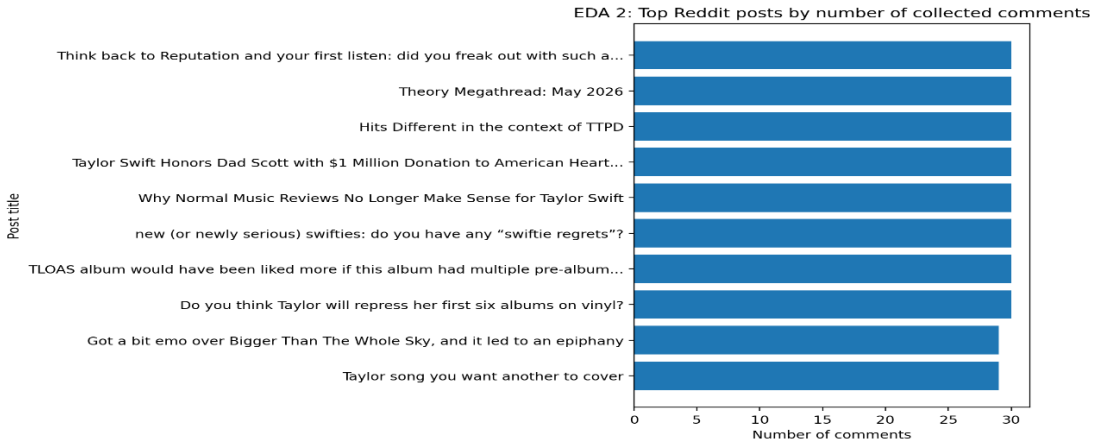


Figure 11: Top posts by comments

The results show that the most active posts involved albums, songs, reviews, vinyl re-recordings, and fan reflections. This indicates that Taylor Swift's Reddit audience deeply explores her artistic choices and albums. For PR, campaign messages should go beyond promotions to include discussions fans enjoy, like album retrospectives, song meanings, fan theories, rankings, and behind-the-scenes stories.

```
# ============================================================
# Exploratory Visual 2: Top Reddit posts by number of comments
# ============================================================

top_posts = (
    df.groupby("post_title")
      .size()
      .sort_values(ascending=False)
      .head(10)
      .reset_index(name="comment_count")
)
top_posts["post_title_short"] = top_posts["post_title"].apply(lambda x: short_label(x, 80))
top_posts.to_csv("outputs/eda_2_top_posts_by_comments.csv", index=False)

plot_data = top_posts.sort_values("comment_count", ascending=True)

plt.figure(figsize=(10, 6))
plt.barh(plot_data["post_title_short"], plot_data["comment_count"])
plt.title("EDA 2: Top Reddit posts by number of collected comments")
plt.xlabel("Number of comments")
plt.ylabel("Post title")
save_current_figure("eda_2_top_posts_by_comments")

display(top_posts)
```

Figure 12: Code used to detect Reddit posts with the highest comments

3.3. EDA 3: Top 20 Frequent Words after Data Cleaning

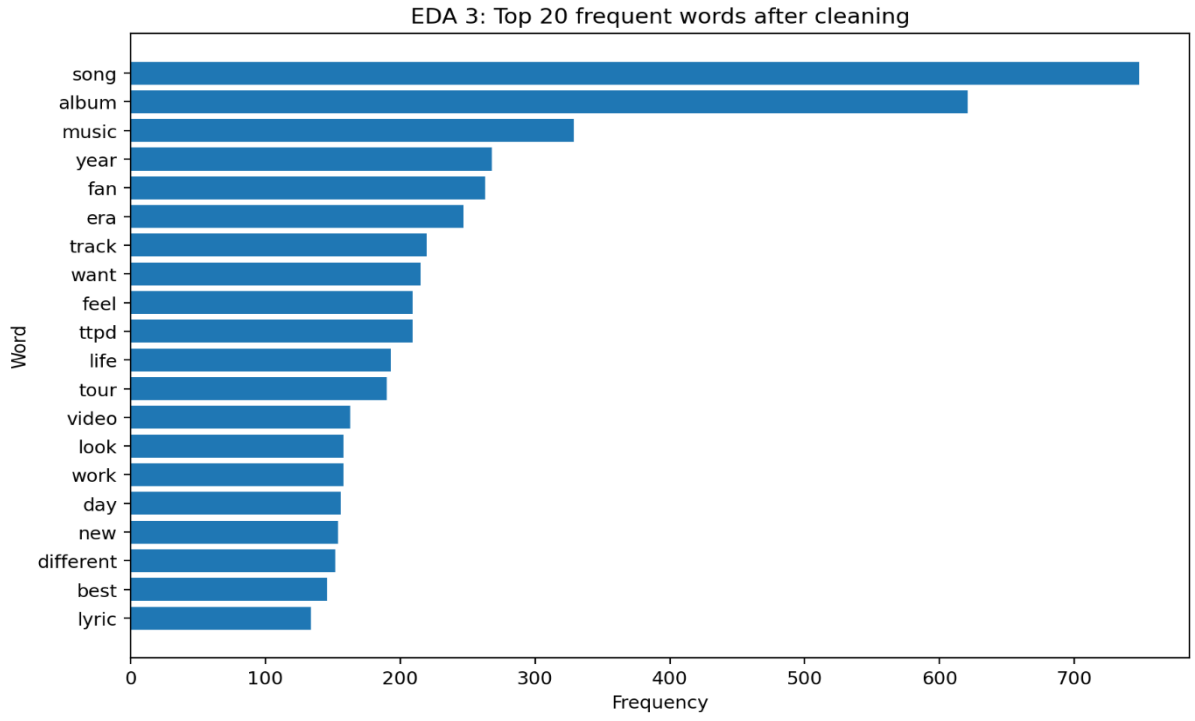


Figure 13: Top 20 frequent words after data cleaning

The figure above shows the most common words after cleaning, stopword removal, and lemmatisation. The recurring terms indicate that the Reddit discussion primarily revolves around Taylor Swift's music and artistic output. The top words, "song", "album", and "music", indicate that the audience focuses on her work, not celebrity gossip. Words like "era", "track", "ttpd", "tour", and "lyric" show fans discuss her creative identity, confirming her brand's strong link to storytelling. PR should continue to highlight her creative work, with campaigns that help fans discuss songs and albums.

```
# ============================================================
# Exploratory Visual 3: Top 20 most frequent cleaned words
# ============================================================

all_tokens = tokens_from_series(df["cleaned_text"])
word_counts = Counter(all_tokens)

top_words = pd.DataFrame(word_counts.most_common(20), columns=["word", "frequency"])
top_words.to_csv("outputs/eda_3_top_20_words.csv", index=False)

plot_data = top_words.sort_values("frequency", ascending=True)

plt.figure(figsize=(9, 6))
plt.barh(plot_data["word"], plot_data["frequency"])
plt.title("EDA 3: Top 20 frequent words after cleaning")
plt.xlabel("Frequency")
plt.ylabel("Word")
save_current_figure("eda_3_top_20_words")

display(top_words)
```

Figure 14: Code used to detect the top 20 frequent words after cleaning

3.4. EDA 4: Distribution of Reddit Comment Length

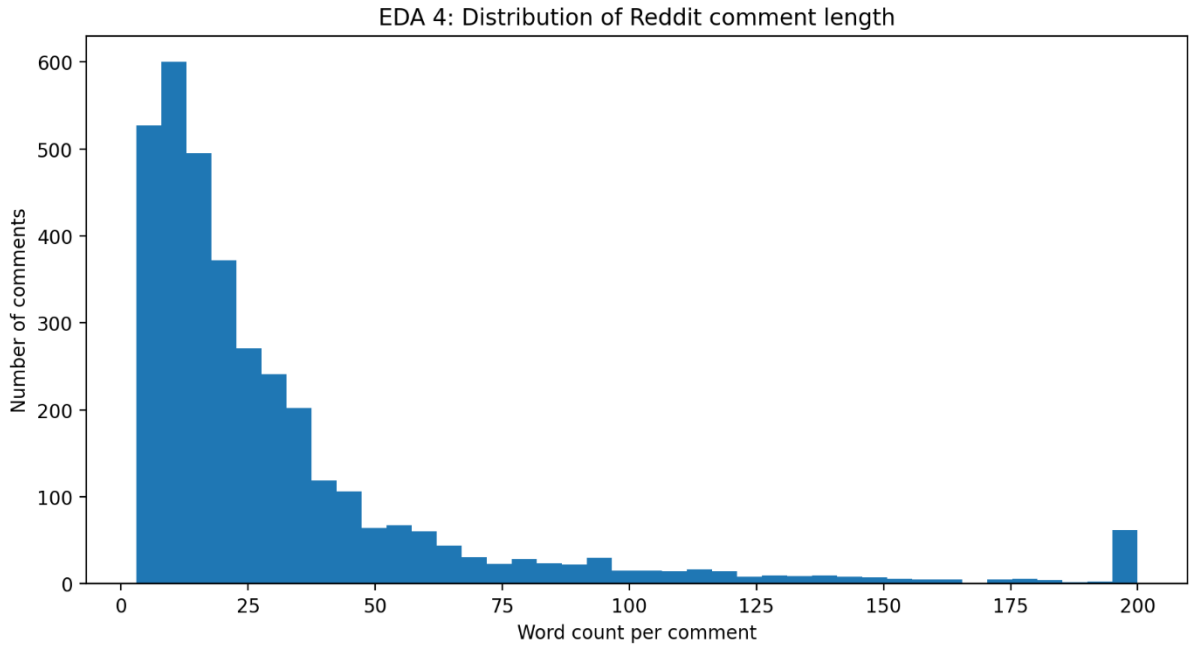


Figure 15: Distribution of comment length

The cleaned dataset contained 3,552 comments, averaging 35.89 words and a median of 19 words. Most comments were short, but a few longer ones raised the average. This reveals two audience behaviours:

1) quick reactions for expressing moods and

2) longer, analytical comments for deeper insights.

Thus, Taylor Swift's Reddit community includes both emotional responses and detailed fan analysis that can help PR teams uncover hidden themes that drive fan engagement.

```
# ============================================================
# Exploratory Visual 4: Comment length distribution
# ============================================================

length_summary = df["raw_word_count"].describe().reset_index()
length_summary.columns = ["Statistic", "Raw word count"]
length_summary.to_csv("outputs/eda_4_comment_length_summary.csv", index=False)

# Clip very long comments at 200 words so the chart is readable.
# The full min/max values remain available in the summary table.
word_count_clipped = df["raw_word_count"].clip(upper=200)

plt.figure(figsize=(9, 5))
plt.hist(word_count_clipped, bins=40)
plt.title("EDA 4: Distribution of Reddit comment length")
plt.xlabel("Word count per comment")
plt.ylabel("Number of comments")
save_current_figure("eda_4_comment_length_distribution")

display(length_summary)
```

Figure 16: Code used to detect the distribution of comment length

3.5. EDA 5: Top 15 Active Anonymised Reddit Authors

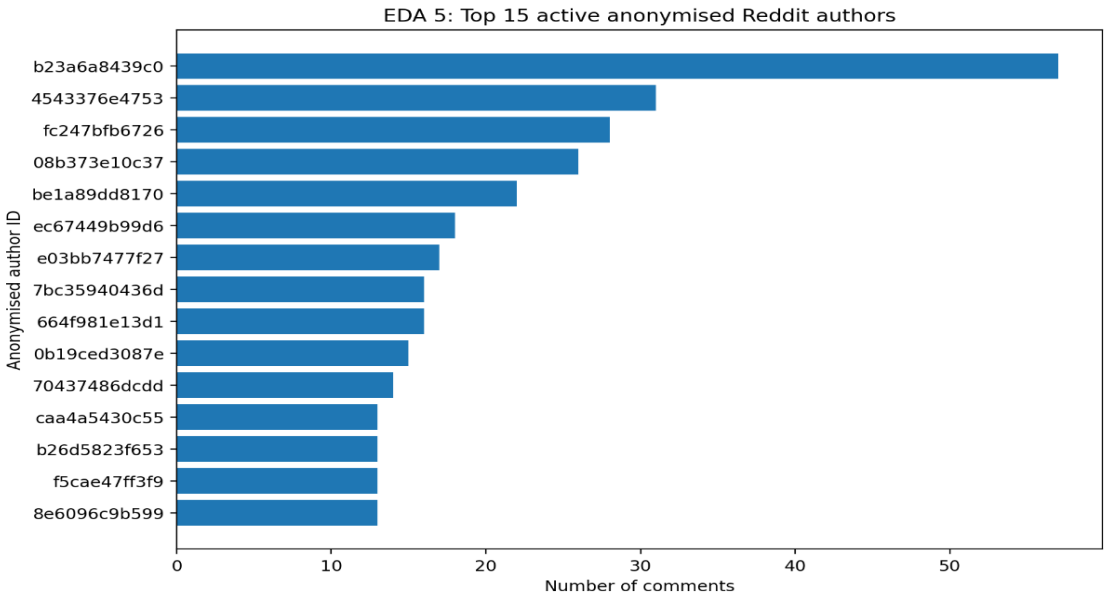


Figure 17: Top 15 active anonymised Reddit authors

The figure above shows that the community is broad, as the top contributor accounted for only 57 comments (about 1.60%) among the 2,016 unique authors. This indicates a conversation thread spanning many contributors rather than dominance by a few users. For PR, this is positive, as insights are less controlled by one person. Thus, PR teams should monitor themes to understand fan interests, rather than target individuals.

```
# ============================================================
# Exploratory Visual 5: Top active anonymised authors
# ============================================================

top_authors = (
    df["comment_author_id"]
      .value_counts()
      .head(15)
      .reset_index()
)
top_authors.columns = ["anonymised_author_id", "comment_count"]
top_authors.to_csv("outputs/eda_5_top_active_authors.csv", index=False)

plot_data = top_authors.sort_values("comment_count", ascending=True)

plt.figure(figsize=(9, 6))
plt.barh(plot_data["anonymised_author_id"], plot_data["comment_count"])
plt.title("EDA 5: Top 15 active anonymised Reddit authors")
plt.xlabel("Number of comments")
plt.ylabel("Anonymised author ID")
save_current_figure("eda_5_top_active_authors")

display(top_authors)
```

Figure 18: Code used to detect the top 15 active anonymised Reddit authors

3.6. Summary and Conclusion

The exploratory analysis revealed five key findings:

- First, Taylor Swift's Reddit activity peaked in March, April, and May 2026.
- Second, most posts focus on songs, albums, reviews, vinyl, TTPD, and Reputation, showing a music-focused audience.
- Third, common words include “song”, “album”, “music”, “fan”, “track”, “ttpd”, and “tour”, indicating discussions about her music and fandom.
- Fourth, comments are mostly short (median 19 words), but some are very detailed, up to 2,715 words, reflecting quick reactions and in-depth analysis.
- Fifth, participation spans 2,016 users, with the most active contributing only 1.60%, indicating broad community involvement.

Overall, the Reddit presence is shaped by music fandom, event-driven activity, diverse participation, and a mix of brief and detailed comments. This perfectly lays the groundwork for sentiment analysis and topic modelling in the next chapter.

4. TEXT MINING

The text mining stage moved from description to the identification of fan sentiment and opinion patterns. It used **VADER sentiment analysis** and **Latent Dirichlet Allocation (LDA) topic modelling**. VADER was chosen for its suitability with social media text (Hutto and Gilbert, 2014), while LDA was selected for its ability to discover hidden themes within text (Blei, Ng and Jordan, 2003). Together, these methods provided useful PR insights from analysing the cleaned dataset of 3,552 comments, revealing how audiences feel and what they discuss.

4.1. Text Mining 1: VADER Sentiment Distribution

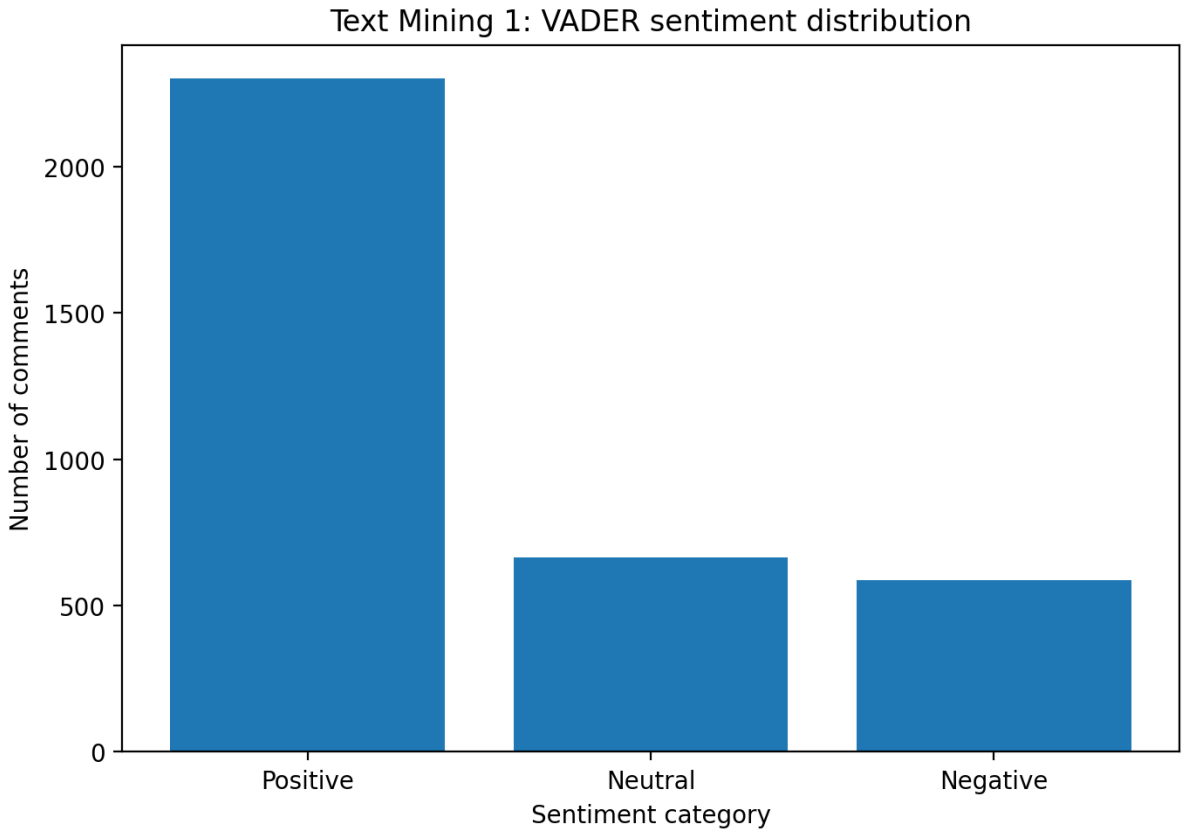


Figure 19: VADER sentiment distribution

VADER sentiment analysis was used to classify comments as positive, neutral, or negative based on the compound score, which ranges from -1 to +1. Comments were positive if the compound score was at least 0.05, negative if the score was -0.05 or below, and neutral if it fell between these values.

The Reddit discussion about Taylor Swift was mostly positive, with nearly two-thirds of comments positive and only 16.55% negative, indicating broad support. This is a strong reputational signal from a PR perspective, showing the community is supportive, engaged, and emotionally invested. However, the 588 negative comments shouldn't be ignored, as they highlight potential areas of criticism, disagreement, or risk.

Below is the sentiment distribution:

Table 4: VADER sentiment distribution

Sentiment	Number of comments	Percentage
Positive	2,300	64.75%
Neutral	664	18.69%
Negative	588	16.55%

```
# ============================================================
# Cell 6: VADER sentiment scoring
# ============================================================

sia = SentimentIntensityAnalyzer()

# Use readable original text where available because VADER handles punctuation, emojis and intensifiers better than heavily cleaned text.
df["sentiment_source_text"] = np.where(
    df["text_readable"].str.len() > 0,
    df["text_readable"],
    df["comment_body"]
)

df["vader_compound"] = df["sentiment_source_text"].apply(lambda x: sia.polarity_scores(str(x))["compound"])

def vader_label(score):
    if score >= 0.05:
        return "Positive"
    elif score <= -0.05:
        return "Negative"
    else:
        return "Neutral"


df["sentiment_label"] = df["vader_compound"].apply(vader_label)

sentiment_summary = (
    df["sentiment_label"]
      .value_counts()
      .rename_axis("sentiment")
      .reset_index(name="comment_count")
)
sentiment_summary["percentage"] = round(sentiment_summary["comment_count"] / len(df) * 100, 2)
sentiment_summary.to_csv("outputs/text_1_sentiment_distribution.csv", index=False)

display(sentiment_summary)

# Save enriched dataset for later report writing and checking examples
sentiment_output_path = "outputs/taylor_swift_reddit_with_sentiment.csv"
df.to_csv(sentiment_output_path, index=False)
print(f"Saved enriched dataset: {sentiment_output_path}")
```

Figure 20: Code used to obtain the VADER sentiment distribution

4.2. Text Mining 2: VADER Sentiment Over Time

The second text-mining visualisation examined whether sentiment changed over time. The sentiment trend remained above zero throughout the data collection period, indicating that comments were generally positive. The highest average sentiment occurred in November 2025, while the lowest occurred in September 2025. For PR interpretation, this suggests that overall sentiment toward Taylor Swift remained consistently positive across the entire period, with only brief dips in September to

October 2025 and in May 2026, suggesting short-lived moments of mixed discussion or heightened debate rather than any sustained shift in audience perception.

Table 5: Average VADER sentiment over time

Month	Average compound sentiment	Comment count
2025-05	0.690	3
2025-06	0.704	8
2025-07	0.714	4
2025-08	0.597	9
2025-09	0.252	7
2025-10	0.443	4
2025-11	0.927	3
2025-12	0.805	3
2026-01	0.762	7
2026-02	0.652	7
2026-03	0.611	25
2026-04	0.601	25
2026-05	0.486	26

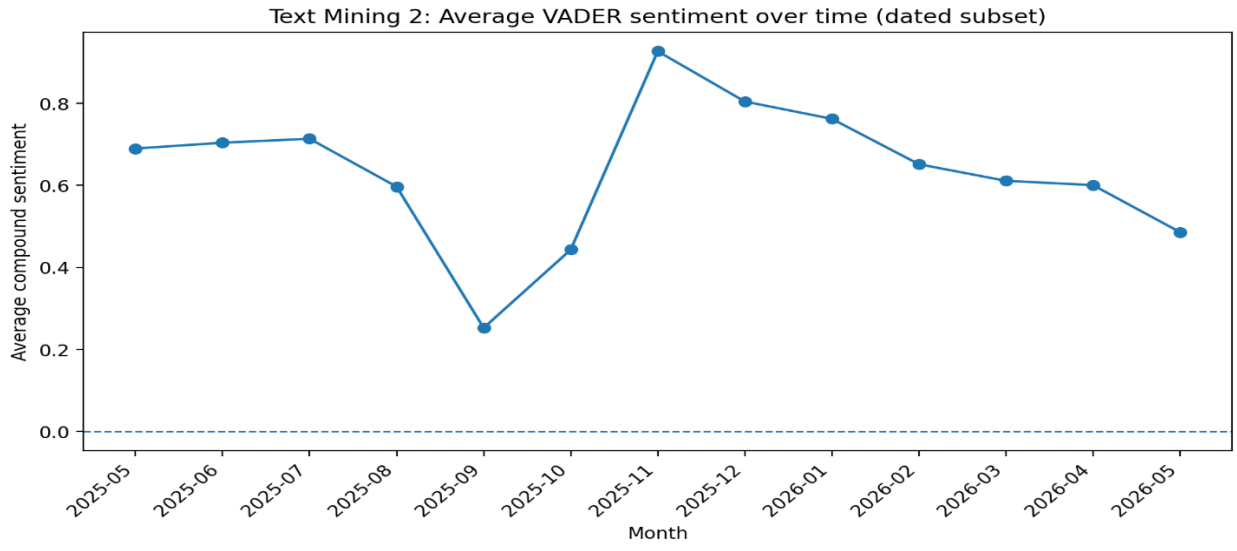


Figure 21: Average VADER sentiment over time

```python
# ============================================================
# Text Mining Visual 2: Sentiment over time
# ============================================================

sentiment_dated = df[df["comment_datetime_parsed"].notna()].copy()

if len(sentiment_dated) > 0:
    monthly_sentiment = (
        sentiment_dated.groupby("comment_month_parsed")
        .agg(
            average_compound_sentiment=("vader_compound", "mean"),
            comment_count=("vader_compound", "size")
        )
        .reset_index()
    )
    monthly_sentiment.to_csv("outputs/text_2_sentiment_over_time.csv", index=False)

    plt.figure(figsize=(9, 5))
    plt.plot(monthly_sentiment["comment_month_parsed"], monthly_sentiment["average_compound_sentiment"], marker="o")
    plt.axhline(0, linestyle="--", linewidth=1)
    plt.title("Text Mining 2: Average VADER sentiment over time (dated subset)")
    plt.xlabel("Month")
    plt.ylabel("Average compound sentiment")
    plt.xticks(rotation=45, ha="right")
    save_current_figure("text_2_sentiment_over_time")

    display(monthly_sentiment)
else:
    print("No parsed dates are available, so sentiment-over-time cannot be created.")
```

Figure 22: Code used to obtain the VADER sentiment over time

4.3. Text Mining 3: Top Terms in Positive and Negative Comments

Table 6: Top terms in positive and negative comments

Positive term	Positive frequency	Negative term	Negative frequency
song	594	song	109
album	507	album	82
music	264	want	47
year	226	point	45
fan	225	tortured	45
era	204	poet	45
track	166	music	45
want	162	track	44
ttpd	160	mean	43

feel	159	department	42

Table 6 compares the most frequent words in positive and negative comments to determine if they discussed different topics or expressed different opinions about the same topics.

Results indicate that positive and negative comments revolve around similar topics, such as songs, albums, music, and tracks. As sentiment differences mainly stem from opinions about Taylor Swift's work, this suggests that conversations focus on her creative output. Also, terms like "tortured", "poet", and "department" in negative comments relate to the album *The Tortured Poets Department*. Since VADER might misinterpret these as negative, the sentiment should be analysed using topic modelling and a qualitative review to ensure accuracy.

```
# ============================================================
# Text Mining Visual/Table 3: Top positive and negative terms
# ============================================================

positive_tokens = tokens_from_series(df.loc[df["sentiment_label"] == "Positive", "cleaned_text"])
negative_tokens = tokens_from_series(df.loc[df["sentiment_label"] == "Negative", "cleaned_text"])

positive_terms = pd.DataFrame(Counter(positive_tokens).most_common(10), columns=["positive_term", "positive_frequency"])
negative_terms = pd.DataFrame(Counter(negative_tokens).most_common(10), columns=["negative_term", "negative_frequency"])

positive_negative_terms = pd.concat([positive_terms, negative_terms], axis=1)
positive_negative_terms.to_csv("outputs/text_3_top_positive_negative_terms.csv", index=False)

display(positive_negative_terms)

# Save the table as an image for easy insertion into the report.
fig, ax = plt.subplots(figsize=(11, 4))
ax.axis("off")
table = ax.table(
    cellText=positive_negative_terms.fillna("").values,
    colLabels=positive_negative_terms.columns,
    loc="center",
    cellLoc="center"
)
table.auto_set_font_size(False)
table.set_fontsize(9)
table.scale(1, 1.4)
plt.title("Text Mining 3: Top terms in positive and negative comments")
save_current_figure("text_3_top_positive_negative_terms")
```

Figure 23: Code used to obtain the top terms in positive and negative comments

4.4. Text Mining 4: LDA Topic Modelling and Discussion Themes

LDA topic modelling was used to identify the main discussion themes in the cleaned Reddit comments. Before selecting the final model, topic models with **3**, **5** and **10** topics were tested. Perplexity was used as a technical guide, with lower values indicating a better statistical fit.

Table 7: Topic models with Perplexity

Number of topics	Perplexity
3	1225.60
5	1266.68
10	1341.65

The 3-topic model had the lowest perplexity score. However, the 5-topic model was selected for the final interpretation because it offered a better balance between detail and interpretability.

```
# ============================================================
# Cell 7: LDA topic modelling - compare 3, 5 and 10 topics
# ============================================================

# LDA works on document-term counts. Bigrams are included to capture phrases such as "era tour" and "music video".
vectorizer = CountVectorizer(
    max_df=0.90,
    min_df=5,
    max_features=5000,
    stop_words=list(CUSTOM_STOPWORDS),
    ngram_range=(1, 2)
)

document_term_matrix = vectorizer.fit_transform(df["cleaned_text"].fillna(""))
feature_names = vectorizer.get_feature_names_out()

lda_results = []
models = {}

for k in [3, 5, 10]:
    lda_model = LatentDirichletAllocation(
        n_components=k,
        random_state=42,
        learning_method="batch",
        max_iter=20
    )
    lda_model.fit(document_term_matrix)
    perplexity = lda_model.perplexity(document_term_matrix)
    models[k] = lda_model
    lda_results.append([k, round(perplexity, 2)])

lda_comparison = pd.DataFrame(lda_results, columns=["number_of_topics", "perplexity_lower_is_better"])
lda_comparison.to_csv("outputs/text_4_lda_model_comparison.csv", index=False)

display(lda_comparison)
print("Note: Perplexity is useful, but the final topic number should also be chosen for interpretability in the report.")
```

Figure 24: Code used to derive the perplexity of the LDA topic models

After the 5-topic model was fine-tuned with the appropriate number of topics, the next step was to visualise the topic themes. The code below generated the bar graph below, which displays the most frequent themes in the cleaned dataset.

```
# ============================================================
# Text Mining Visual 4: Dominant topic distribution
# ============================================================

topic_distribution = (
    df["dominant_topic_label"]
    .value_counts()
    .reset_index()
)
topic_distribution.columns = ["topic_label", "comment_count"]
topic_distribution["percentage"] = round(topic_distribution["comment_count"] / len(df) * 100, 2)
topic_distribution.to_csv("outputs/text_6_dominant_topic_distribution.csv", index=False)

plot_data = topic_distribution.sort_values("comment_count", ascending=True)

plt.figure(figsize=(10, 6))
plt.barh(plot_data["topic_label"], plot_data["comment_count"])
plt.title("Text Mining 4: Dominant discussion themes from LDA topic modelling")
plt.xlabel("Number of comments")
plt.ylabel("Dominant topic label")
save_current_figure("text_4_dominant_topic_distribution")

display(topic_distribution)
```

Figure 25: Code used to visualise the most frequent themes

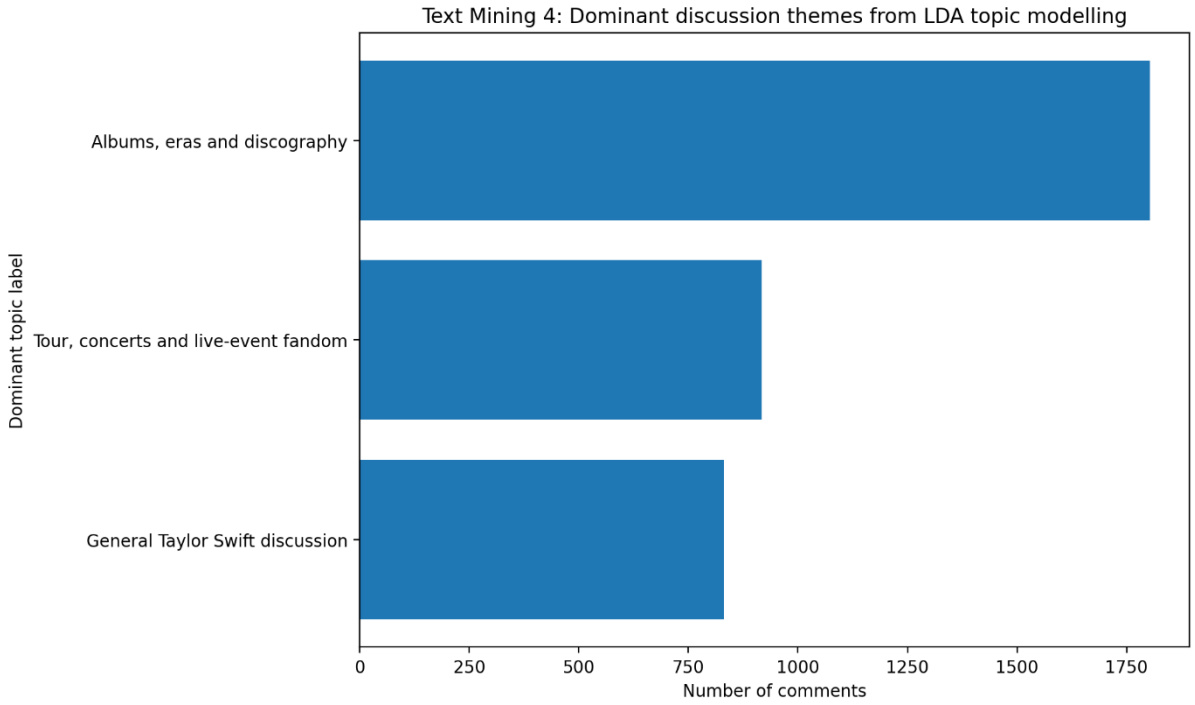


Figure 26: Dominant discussion themes from LDA topic modelling

The main themes were albums, eras, and discography, with 1,803 comments (50.76%), indicating that discussions about her music and artistic identity dominate her Reddit presence. The second theme was tours, concerts, and live events, with 917 comments (25.82%), highlighting fan engagement with live performances and ticket purchases. The third was a general discussion of Taylor Swift, with 832 comments (23.42%) covering opinions, personality, and public image. These findings suggest that her Reddit audience centres on her music catalogue, live events, and broader identity, informing a PR strategy that emphasises her creative work, tour excitement, and fan engagement.

4.5. Text Mining 5: VADER Sentiment by LDA Topic

The final text mining analysis combined sentiment analysis with topic modelling. This is one of the most useful outputs for PR decision-making because it shows not only what people are discussing, but also how positive or negative they are within each topic.

Table 8: Average VADER sentiment by LDA topic

Dominant topic label	Comment count	Average compound sentiment	Positive comments	Neutral comments	Negative comments
Albums, eras and discography	1,803	0.332	1,183	318	302
General Taylor Swift discussion	832	0.264	483	194	155
Tour, concerts and live-event fandom	917	0.382	634	152	131

All three discussion themes had positive sentiment, showing that Taylor Swift's Reddit audience is broadly favourable.

- **Tour, concerts, and live-event fandom** were the most positive, with a sentiment of 0.382 and about 69.14% positive comments.
- **Albums, eras, and discography** had an average of 0.332, and the most comments.

The PR insight is that tour and live-event discussions generate the most positive sentiment, while album and discography topics drive engagement. A successful PR strategy should combine these: use albums and discography to increase discussion volume, and tour and event content to enhance emotional connection.

4.6. Summary and Conclusion

The text mining shows that Taylor Swift's Reddit discussions are driven by emotional reactions, fan theories, and themes surrounding album release or events. This indicates that her fans actively engage with her work, so PR teams should monitor patterns, understand fan interpretations, and watch for tone shifts that signal opportunities or issues.

5. CONCLUSION

5.1. Critical Evaluation of Key Results and Insights

The analysis reveals that Taylor Swift's Reddit presence is highly active, positive, and centred around her music, albums, tours, and fan experiences. The key PR insight is that her strongest reputation driver on Reddit is the emotional bond fans have with her music and live events, rather than her general celebrity visibility. This was supported by frequent music-related terms in the Reddit comments, positive VADER sentiment, and topic modelling, indicating that albums, discographies, tours, and live-event fandom dominated the discussion. Therefore, PR campaigns should focus on music storytelling, album engagement, live-event nostalgia and fan discussions, while monitoring broader celebrity conversations for reputational risks.

5.2. Critical Evaluation of the Adopted Approach

The adopted approach was effective because it combined exploratory analysis, sentiment analysis and topic modelling to examine both what Reddit users discussed and how they felt. Using Reddit RSS feeds was practical and ethical because the data was publicly accessible and did not require the additional step of using API credentials. However, the VADER analysis showed limitations in its interpretations of fandom language and album titles such as *The Tortured Poets Department*. Nevertheless, the VADER Sentiment by LDA Topic findings addressed this limitation and provided the most useful PR insights for analysing Taylor Swift's presence on Reddit.

5.3. Critical Reflection on Skills Developed and Experience Gained

This work developed my practical understanding of the social media analytics process, from data collection and preprocessing to modelling, visualisation and interpretation. I gained experience cleaning noisy Reddit text, creating summary statistics, building visualisations, applying VADER sentiment analysis, and using LDA topic modelling to identify audience themes. I also developed stronger critical judgment skills by recognising how to overcome the limitations of sentiment models and topic modelling. Overall, the project improved my ability to extract and turn unstructured social media data into meaningful, evidence-based insights for PR decision-making.

REFERENCES

Blei, D.M., Ng, A.Y. and Jordan, M.I. (2003) 'Latent Dirichlet allocation', *Journal of Machine Learning Research*, 3, pp. 993–1022.

Hutto, C. and Gilbert, E. (2014) 'VADER: A parsimonious rule-based model for sentiment analysis of social media text', *International AAAI Conference on Web and Social Media*.